REVERSE ENGINEERING · APPLE POWER ACCESSORIES

Apple USB-C and MagSafe 3 power path

Three processors, three firmware objects, one name

The cable's three releases extracted, the container format recovered from Apple's own parser, and the cross-version facts established without touching a device.

REPORT	VERSION	DATE	CLASSIFICATION
RR-2026-1114	1.1	2026-11-11	Internal
TARGET	BASELINE	PREPARED BY	TEMPLATE
3.2.0	3.1.8	Sein	v1.2.0

DOCUMENT CONTROL

REPORT	Apple USB-C and MagSafe 3 power path
REPORT ID	RR-2026-1114
VERSION	1.1
DATE	2026-11-11
CLASSIFICATION	Internal
BASELINE	MagSafe 3 · 3.1.8
TARGET	MagSafe 3 · 3.2.0
PREPARED BY	Sein
SOURCE MATERIAL	Apple public UARP assets, hpmdiagnose register dumps, IORegistry captures
RENDERER	Typst · Research Reports template 1.2.0

CONTENTS

1 Executive summary 3
2 The three objects 4
3 The host power path 5
4 Acquiring the firmware 5
5 Reading the cable images 6
6 What every build shares 6
7 The dispatcher story 7
8 Diffing at function level 7
9 Integrity and crypto 8
10 Update transport 9
11 Security posture 9
12 Limits 10
13 Method 10
14 Appendix A — claim register 11
15 Appendix A — claim register 11

1 Executive summary

Three firmware objects share the phrase “the charger firmware” and nothing else. This report separates them on a live Apple Silicon MacBook Pro, extracts the MagSafe 3 cable’s three public releases, recovers the container format from Apple’s own parser, and then rebuilds the analysis of the cable images themselves: a Cortex-M function inventory that reproduces byte-identically, a decoded integrity header, a calibrated cross-version matcher, and a per-pair instruction-level diff — read-only throughout.

VERDICT

The host power path, the container format and the execution model are established. The cable image’s integrity header is fully decoded and reproduces on every build. Between 3.1.12 and 3.2.0, 124 of 155 confidently matched function pairs are identical once relocation is masked; 31 differ at the instruction level, and one of those differences — a signed-to-unsigned comparison change — is shown instruction by instruction. Where a question cannot be closed on static evidence, it is named with the observation that would settle it.

KEY FINDINGS

01 **The integrity header is a hash chain** — The opaque 384-byte `C99` header decoded into two records whose SHA-256 and CRC-32 fields cover the shipped image, the header plus image, a second crypto-bearing module, and the header itself. Every relation reproduces byte-exactly on all three releases.

02 **Most of the code did not change** — A matcher calibrated against within-build hard negatives resolves 155 function pairs between 3.1.12 and 3.2.0 at a false-positive rate of zero; 124 of them are identical once branch and call displacements are masked. 31 pairs differ substantively.

03 **The dispatcher story, corrected** — The celebrated 14-case control-message table exists in both 3.1.x builds and has no detected 3.2.0 counterpart — but its island has zero static inbound edges and even table entries that would fault on v7-M. It is most likely a stale copy, and the live handler's location is open.

04 **Crypto is present, not absent** — SHA-256 round constants and a complete secp256r1 parameter block sit in every image. The earlier reading that the image carries no cryptographic material was an artifact of the header being stripped.

3	444/436/423	46/45/56 %	155
CABLE RELEASES EXTRACTED	FUNCTIONS RECOVERED PER BUILD	OF IMAGE INSIDE ANALYSED FUNCTIONS	CALIBRATED FUNCTION PAIRS ACROSS VERSIONS

2 The three objects

Calling all of this "charger firmware" is what makes the topic confusing. They are three different processors with three different update paths.

TABLE 1

Firmware objects in the power path

OBJECT	IDENTIFIER	HOW IT WAS READ
HPM controller	`FW003.115.00`	From the kext's own strings and register file
MagSafe 3 cable	3.2.0 / PID `0x7800` / `0x3200`	Register `0x49` and the in-file version

OBJECT	IDENTIFIER	HOW IT WAS READ
70 W adapter	`1030030` / ID `0x701c`	Host-reported only: no adapter firmware obtained

Source: IORegistry and `IOAccessoryManager` captures in `evidence/`, `hpmdiagnose` register dumps, and `AppleHPM` decompilation.

The distinction matters for any security claim. The host can read the cable's version out of the negotiation it already performs; it cannot read the adapter's flash, and nothing in the acquired data shows otherwise.

3 The host power path

Four HPM instances exist: three USB-C ports at RID 0, 1, 2 and the MagSafe port at RID 5. The charger is on RID 5, and the winning power option is 20 V at 3.390 A, which is 67.80 W.

ACCESS

The full HPM register file is readable without root by `hpmdiagnose`; access is gated by the `com.apple.USBCEntitlement` entitlement, not by privilege. That is the practical ceiling of noninvasive inspection, and it is higher than the usual "needs root" assumption.

What the host exposes is a contract: a set of fixed power options, one of which is selected. That is a negotiation result, not a firmware property, and the two should never be tabulated as the same kind of fact.

4 Acquiring the firmware

Apple publishes accessory firmware under `com.apple.MobileAsset.UARP.<model>` on `updates.cdn-apple.com`. Each download was verified against the SHA-1 Apple itself publishes in `\_Measurement`, before anything was unpacked.

TABLE 2

Acquired cable releases

ASSET	SHA-1 (TRUNCATED)
A2363_magsafe3_3.1.8_10M569.zip	3722c634e671…
A2363_magsafe3_3.1.12_10M1543.zip	a10fd2efd5cc…
A2363_magsafe3_3.2.0_10M13722.zip	4db70dec4835…
A2452_140W_1.4.90_10M13554.zip	60b42b01ce02…

Source: `firmware/MANIFEST.txt`; SHA-1 truncated here, full values and URLs in the manifest.

The 70 W adapter is the exception, and the negative is scoped rather than absolute: three enumeration methods failed — `mesu` path probes, a Pallas lookup including a routine step. Whether that verb can touch flash is `OPEN`.

5 Reading the cable images

The three releases are flat ARM Thumb images. Earlier analysis ran them through Ghidra's generic ARM profile, which mis-delimits Thumb-2 code and left the interesting regions outside every function body. The rebuilt pipeline fixes the language (`ARM:LE:32:Cortex`, the Thumb-only Cortex profile), then earns its function boundaries instead of trusting the analyzer: every candidate entry — vector words, dispatcher case targets, prologue candidates from an offline branch cross-reference map — is diagnosed first (what listing unit sits there, what references it, does it decode), and only verified code becomes a function. Where a wrong data unit sits on decodable code, the clear is precise, gated on the recorded diagnosis, and logged with before-and-after evidence.

TABLE 3

Recovered function inventories

BUILD	FUNCTIONS	COVERAGE	PREVIOUS PROFILE
3.1.8	444	46.4 %	300 / 38.8 %
3.1.12	436	45.2 %	303 / 39.4 %
3.2.0	423	56.2 %	296 / 43.3 %

Source: seeded Ghidra runs, `evidence/magsafe-cortexm/inventory3_318.json` and the delegated runs for the other builds; two independent 3.1.8 runs produce identical inventories.

Three boundaries hold throughout. Addresses are file offsets: the images are imported at base zero for analysis, and no runtime base is asserted. The startup candidate at offset `0x10` decodes as a clean Thumb prologue in all three builds, but the vector-table words at the image head point into a constant table and decode as nothing — so `0x10` stays a candidate, not a proven reset entry. And roughly a quarter of each image is coherent code with no static caller: islands that either run through a mechanism invisible to static analysis or are dead.

6 What every build shares

The container, extracted with Apple's own parser, yields a flat image whose payload body **is** the analysed file: the extracted image equals the payload minus its first 384 bytes. Those 384 bytes turned out to be the most informative part of the file — see the integrity section below.

Content that moves between builds is layout, not logic. The power-negotiation table (5, 9, 15 and 20 V at 3 A, a fixed PDO list) is byte-identical in all six copies across all three releases while its address moves. No ASCII version string exists in any image;

versions are numeric and rendered by the host. Of the Apple SVID literals, exactly one changes across the releases: `0x05AC0002` disappears and `0x05AC0000` gains an occurrence — a real byte-level fact whose meaning stays open, because the enclosing functions were outside the earliest analysed set.

---

7 The dispatcher story

The 3.1.x builds contain a 14-case dispatch table over USB-PD message types 9 through 22, reached through a `subs r0,#9; cmp r0,#0xd` guard and a `mov pc` computed branch. Seeding recovered its enclosing function: 552 bytes by Ghidra's boundary, 796 by real control flow. Its six unique case targets, its table boundaries (the word after the fourteenth entry is not a pointer), and its out-of-table cases (2, 3, `0x28`, `0x3d`, `0x41`) are all verified at the instruction level.

Then the story turns. The island containing all of this has **no** static inbound edge of any kind — no call, no wide branch, no pointer word, no halfword. Three of its case targets enter foreign stack frames. And its table entries are even addresses consumed by `mov pc`: on v7-M an even value there requests ARM state, which would fault. The most economical reading is that this island is a stale or dead copy. The dispatch **mechanism** is real and used elsewhere — reachable code installs a 13-entry table of runtime-filled function pointers at a per-port object and calls through it 47 times — but the islands are never its target.

CORRECTION HISTORY

This section was rewritten three times under audit. The switch site called 0xAA2A was actually 3.1.12's; the decompiler's dataflow claim at 0xaf76 survived two failed refutations because the register in question carries the table base on the table path and an argument on the direct-call path. Both corrections are recorded in DIFF.md rather than silently applied.

The second stage of the dispatcher does have a clean lineage. A 17-case payload switch sits at `0xad2a` in 3.1.8, `0xad7e` in 3.1.12 and `0xab64` in 3.2.0 with an identical guard (`mov r3,r2; subs r3,#8; cmp r3,#0x10; bls`), identical index chain and table fetch, and per-case grouping that matches build for build. The only structural change is the index width: 3.1.x reads a byte table, 3.2.0 reads a halfword table. That is an inferred-but-strong lineage, and it survives while the outer 14-case table does not — 3.2.0 instead shows three unrelated switches with different guards and index sources, none of which matches the 3.1.x dispatcher on shape or callee set.

---

8 Diffing at function level

Byte-level comparison misleads here: only 4 % of 3.2.0's bytes sit at the same offset as 3.1.12's, yet the builds share most of their code — a relink moves almost every instruction that carries an address. The matcher therefore works on normalized mnemonic sequences, and — the part that failed twice before it worked — calibrates

against **within-build** hard negatives: near-duplicate helpers inside the same image, the exact class that once drove the threshold to 1.0 and voided two earlier runs. The chosen threshold of 0.983 sits just above the hardest real negative (0.9829) and far below 1.0; on 352 known-true cross-build pairs from the 3.1.8-to-3.1.12 comparison it scores a true-positive rate of 1.0.

TABLE 4

The 155 confidently matched 3.1.12-to-3.2.0 function pairs, classified from decoded instructions

CLASS	PAIRS	READING
Identical modulo relink	124	Branch/call displacements differ; bodies are the same
Field-offset change	20	Struct offsets moved — layout growth, e.g. a record stride +7
Immediate change	4	Constants differ; individual readings inferred case by case
Register allocation	2	Same operations, different registers
Sequence edit / reordering	3	Instructions added, removed or reordered
Signedness change	1	Signed comparisons became unsigned — see below
Indexed load replaces fixed	1	A fixed-offset read became table-indexed

Source: `evidence/magsafe-cortexm/delegated/semantics/results_pair_class.json`; every class backed by decoded windows in both images.

One further pair was excluded before classification: its tail diff was a decoder desync, confirmed by re-decoding from a later offset.

The headline pair is small enough to check by eye: at `0xa018` in 3.1.12 and `0xa020` in 3.2.0, the code compares `r4` against `r0` and branches `ble`/`bgt` in the old build, `bls`/`bhi` in the new one. Same instruction positions, same operands, different comparison semantics — signed versus unsigned. The matcher scores this pair a perfect 1.0, because its normalisation collapses every conditional branch to one token; the difference surfaced only in the per-pair decode pass. That is recorded as a property of the matcher, not hidden as an embarrassment.

9 Integrity and crypto

The 384 bytes stripped during extraction are a two-record hash chain, and every relation in it reproduces byte-exactly on all three payloads.

TABLE 5

The C99 header as a hash chain — all relations independently reproduced

FIELD	COVERS	VERIFIED
`0x110` / `0x108`	SHA-256 / CRC-32 of the shipped image body	all three builds
`0x0c`	SHA-256 of header prefix plus body (record 3)	all three builds

FIELD	COVERS	VERIFIED
`0x30`	SHA-256 of the trailing record 4 (a second module)	all three builds
`0xe0`	SHA-256 of the header's own first 160 bytes	all three builds

Source: recomputed with Python hashlib/zlib on all three payloads; scripts and logs in `evidence/magsafe-cortexm/delegated/security/`.

Inside the images, SHA-256 round constants and initialization vectors appear twice, and a complete secp256r1 parameter block — prime, curve coefficient, generator coordinates and group order — sits byte-exact in all three builds. No RSA, AES or secp256k1. The 64 header bytes at `0xa0..0xdf` are range-valid as an ECDSA signature pair but are not on the curve: the embedded-public-key reading is refuted, and what they actually are stays open.

The largest function that changes across the builds (`0xd904` and its successors, roughly two kilobytes) is a table-driven flash programmer: 18 address/value records, transfers split at 256-byte boundaries, command and address bytes written to a fixed port with per-byte status polling, erase commands at 4-kilobyte alignment, all guarded to a 256-kilobyte address ceiling. It contains no hashing and no comparison — so whatever enforces the hash chain is not this function, and nothing observed yet wires the two together.

---

10 Update transport

On the host, `VDMs` is not a frame format but a 4-character command tag, one tag among many that `AppleHPM`'s command path can carry; the firmware-update path uses `CFUa`, `CFUp` and `CFUs` as runtime arguments, which is why those strings appear nowhere in the kext binary. On the cable side, all three builds contain a real vendor-defined-message dispatcher: it compares the top half of the VDO against Apple's SVID `0x05AC`, splits structured from unstructured messages, and handles one to five VDOs — with no 4-character-tag comparison anywhere. The update payload's road into the cable is therefore not visible above that layer in these images; whether it is consumed below the application image (in the PD PHY or ROM) is open, and a single passive CC-line capture during a real update would settle the endpoint, the envelope and the chunk size at once.

---

11 Security posture

What can be stated: the host verifies the asset hash before applying anything, and the accessory image carries a complete SHA-256/CRC-32 hash chain over itself, a second module and its own header. What cannot: whether a signature is enforced below that chain, whether anti-rollback exists (no version threshold constant or wide compare exists in any image, but at roughly half coverage that negative is bounded, not conclusive), and where the hash chain is checked. Nothing here is reported as a vulnerability.

12 Limits

The function inventories cover roughly half of each image; the uncovered remainder includes a large coherent code island with an `NVM0`-tagged descriptor table and no external caller — the same unreachable-island pattern as the dispatcher. The live PD handler has not been located. The 30 lower-confidence function pairs are unvalidated, and the one validator tried for them was confounded by pointer churn and recorded as a negative result. The adapter firmware was never obtained, so adapter behaviour is host-observed only.

WHAT WOULD SETTLE THE REST

A passive CC-line capture during an update settles transport endpoint and envelope. A runtime image dump via the accessory's debug read verb — which needs its own approval gate — would relocate every island and likely close the liveness question. Anti-rollback testing requires offering an older build to a newer device, which is blocked without a recovery story. Everything else on the open list is static work in the uncovered islands.

13 Method

Evidence came from IORegistry and `IOAccessoryManager` captures, a deterministic `hpmdiagnose` dump, Apple's public manifests, and static analysis. One label scale throughout: `OBSERVED`, `REPRODUCED`, `INFERRED`, `UNVERIFIED`, `REFUTED`, `OPEN`. Subagent investigations wrote disjoint evidence directories with incremental findings; a separate reviewer reproduced the load-bearing claims from clean imports before reading the implementers' conclusions, and both refutations and refuted refutations are on the record in `DIFF.md`.

SCOPE

All hardware interaction was read-only: no flashing, no update trigger, no accessory command. Proprietary images are not redistributed; hashes and acquisition notes are.

14 Appendix A — claim register

15 Appendix A — claim register

Every claim is checked across the research documents by `scripts/verify_claims.py`, which fails on a missing statement, a conflicting label in the same paragraph, a withdrawn phrase in a conclusion, or a broken evidence reference.

- **C01** [OBSERVED] The charger negotiates 20 V at 3.390 A (67.80 W) on the MagSafe port.
- **C02** [OBSERVED] Four HPM controllers are present at RID 0, 1, 2 and 5; RID 5 carries the MagSafe port.
- **C03** [OBSERVED] HPM controller firmware FW003.115.00, the MagSafe cable at 3.2.0 and the adapter value 1030030 are three distinct objects, not revisions of one image.
- **C04** [REPRODUCED] The UARP SuperBinary container is a 44-byte header with 40-byte payload records; dataOffset plus dataLength reconciles in all four files.
- **C05** [OBSERVED] Three MagSafe 3 releases were acquired and each SHA-1 matches Apple’s manifest _Measurement.
- **C06** [REPRODUCED] The power-negotiation table content (5/9/15/20 V at 3.00 A) is identical in all builds while its address moves.
- **C07** [OBSERVED] Exactly one Apple SVID literal changes across the three releases: 0x05AC0002 disappears and 0x05AC0000 gains an occurrence.
- **C08** [REPRODUCED] Between 3.1.12 and 3.2.0 the compared bodies share 11 non-trivial literals, lose 7 and gain 1; 28 is present only in 3.2.0.
- **C09** [REFUTED] The vector word at 0x0001111d points to 0x1111c, which is constant data and not a reset handler.
- **C10** [OBSERVED] The 140 W control image’s pointers 0xec89 and 0xec71 exceed its 0xea00 payload, which rules out a base-zero mapping only; a non-zero base such as 0x400 would place them inside.
- **C11** [REPRODUCED] The frame header formula (sop<<4)|ceil(bytes/4) reproduces the observed 0x21 for SOP selector 2 and a 4-byte payload; the meaning of the sop field outside the observed selector set is open.
- **C12** [OBSERVED] A 14-case mov-pc dispatch table with 6 unique targets and 8 of 14 cases to one target is present in both 3.1.x builds and is not detected in 3.2.0.
- **C12b** [OPEN] The 15-case and 13-case tables in 3.2.0 are candidates only; no correspondence to the 3.1.x switch is shown.
- **C13** [REFUTED] Claims that 3.2.0 replaced or reorganised the dispatcher, or is a rebuild rather than a patch, are withdrawn.
- **C14** [UNVERIFIED] The Apple SVID literal sites are not established as a UVDM builder; that identification remains unverified.
- **C15** [REFUTED] The 5.9 % / 73.1 % aligned-window overlap figures measure layout; the inference from them to a rewrite is refuted.

- **C16** [REPRODUCED] Ghidra function coverage is 38.8 / 39.4 / 43.3 % of image bytes, and the switch and literal sites lie outside the analysed set.
- **C17** [REFUTED] Hard-reset handling change is refuted: Hard_Reset has no MessageType, so a 9-22 message-type switch cannot speak to it.
- **C18** [OPEN] No soft-to-hard reset escalation ladder was found; the question stays open across all three images.
- **C19** [OBSERVED] 70 W adapter firmware was not found by three enumeration methods (mesu path probes, Pallas lookup with a control type, and a broad type sweep); this is a scoped negative, not proof of nonexistence.
- **C20** [OPEN] Signature enforcement below the manifest hash, anti-rollback, and whether the Mode 7 read verb can touch flash are all open.
- **C21** [OBSERVED] hpmdiagnose reads the full HPM register file unprivileged; access is entitlement-gated, not root-gated.
- **C22** [OPEN] Semantic equivalence of 3.2.0 to 3.1.12 is undeterminable at the achieved coverage and is open.
- **C23** [OPEN] The entry-point question is OPEN: offset 0x10 is the empirical start, not a proven entry, and 0x1111c holds peripheral addresses.
- **C24** [REFUTED] The 140 W control image does not corroborate a base-zero layout for the MagSafe images.
- **C25** [OBSERVED] The three firmware objects are not equally established: the adapter value is host-reported only, and the cable version is a single in-file source consistent with register 0x49.
- **C26** [REPRODUCED] 3.2.0 contains 10 detected mov-pc dispatch sites, not 15.
- **C28** [REPRODUCED] The C99 inner header is a two-record SHA-256/CRC-32 hash chain over the payload body, record 3, record 4, plus a header self-hash; all five relations reproduce byte-exactly on all three payloads.
- **C29** [REPRODUCED] The flat base0 images equal payload[0x180:] — the 384-byte C99 header is stripped in extraction, not absent from the asset.
- **C30** [REPRODUCED] SHA-256 (FIPS 180-4 K table + IV, twice per image) and a complete secp256r1 parameter block (p, b, Gx, Gy, n) are present in all three images.
- **C31** [INFERRED] The 3.1.x dispatcher's second-stage 17-case switch has a verified 3.2.0 successor (0xad2a/0xad7e/0xab64) whose only structural change is the index width (ldrb -> ldrh).
- **C32** [OPEN] The 3.1.x dispatcher island has zero static inbound edges and even mov pc table targets (ARM-state request on v7-M); it is very likely a stale or dead copy, and the live dispatcher's location is open.
- **C33** [OBSERVED] Of the 155 non-stub calibrated 1:1 pairs 3.1.12->3.2.0, 124 are identical modulo relink and 31 are substantive, including a signed-to-unsigned comparison change (0xa018->0xa020).
- **C34** [OBSERVED] The 3.1.x dispatcher default arm (including Soft_Reset, case 13) rebuilds the source-cap buffer and sets a port state byte; no Accept transmit or CC/PHY write is decoded in that body.
- **C35** [OBSERVED] The FSM state-name string spm_host_src_hardreset exists in 3.1.8 and 3.1.12 and is absent from 3.2.0, whose pool retains four reordered states.
- **C36** [OPEN] No version-threshold constant or cmp.w instruction exists in any image; at 46-56% function coverage this negative is bounded, not conclusive — anti-rollback is unresolved.

- **C37** [OBSERVED] VDMs is one 4CC command tag among many; UARP uses CFUa/CFUp/CFUs as runtime 4CC arguments. The cable images contain an SVDM dispatcher (0xb34c) with no 4CC compare; CFU consumption below the app image is open.
- **C27** [OBSERVED] Literal-multiset agreement cannot validate the 30 lower-confidence cross-version function pairs: function literal pools are dominated by image and SRAM pointers that relocate wholesale, so the measured disagreement does not distinguish logic change from relink. Those pairs remain unvalidated.